# Отчёт по лабораторной работе №2

Выполнил

Урванцев Дмитрий Алексеевич

6214-100503D

# 1. Результат выполнения первой части задания

Python:

Сгенерированная случайная последовательность:
1001111000101001010100001011001110101110101000010111100010001000011000010000011101110001000011100110110110110111010101100100011110

Код:

```python
def generate_string(length: int) -> str:
    """
    Функия для генерации псч
    """
    result = ""
    random.seed(time.time())
    for i in range(length):
        rand_int = random.randint(0,255)
        rand_int_8bit = "0"*(8-len(bin(rand_int)[2:]))+bin(rand_int)[2:]
        result += rand_int_8bit
    return result
```

C++:

Сгенерированная случайная последовательность:
0000100110110110000001000100111001001001000011101011101111111001010010110000000110001110011101110001000001001000101110000110000 1

Код:

```cpp
void generate_string()
{
    srand(time(NULL));
    std::ofstream file("cpp_string.txt");
    if (file.is_open())
    {
        for (int i = 0; i < 128; i++)
        {
            int random_bit = rand() % 2;
            file << random_bit;
        }
    }
    else
    {
        std::cout << "Не удалось сгенерировать последовательность, увы(";
    }
}
```

Java:

Сгенерированная случайная последовательность:
1111010000101010101001101001101111100110011111111001101111010011111111000111010111000000101010000111111010100001101101100101001 0

Код:

```java
public class prng_java {
    public static void main(String[] args) {
        Random random = new Random();
        StringBuilder bits = new StringBuilder(128);

        // Генерируем последовательность из 128 бит (0 или 1)
        for (int i = 0; i < 128; i++) {
            bits.append(random.nextInt(2));
        }

        // Записываем результат в файл
        try {
            Files.writeString(Path.of("java_string.txt"), bits.toString());
            System.out.println("Успешно: 128 бит записаны в java_string.txt");
        } catch (IOException e) {
            System.err.println("Ошибка записи в файл: " + e.getMessage());
        }
    }
}
```

# 2. Результат выполнения второй части задания

Коды тестов:

Частотный побитовый тест

```python
def bit_frequency_test(bitstring: str) -> float:
    """
    Функция для частотного побитового теста
    """
    sumn = (bitstring.count('1')-bitstring.count('0'))/math.sqrt(len(bitstring))
    pvalue = math.erfc(abs(sumn)/math.sqrt(2))
    return pvalue
```

Тест на одинаковые подряд идущие биты

```python
def consecutive_bits_test(bitstring: str) -> float:
    """
    Функция для теста на одинаковые подряд идущие биты
    """
    pvalue = 0
    xi = bitstring.count('1')/len(bitstring)
    if (abs(xi-0.5) < (2/math.sqrt(len(bitstring)))):
        vn = 1
        for i in range(len(bitstring)-1):
            if (bitstring[i]!=bitstring[i+1]):
                vn += 1
        pvalue = math.erfc(abs(vn-2*len(bitstring)*xi*(1-xi))/(2*math.sqrt(2*len(bitstring))*xi*(1-xi)))
    return pvalue
```

Тест на самую длинную последовательность единиц в блоке

```python
def longest_sequence_test(bitstring: str) -> float:
    """
    Функция для теста на самую длинную последовательность единиц в блоке
    """
    pvalue = 0
    blocks = [""]*16
    for i in range(0,8):
        blocks[0]+=(bitstring[i])
    for i in range(8,len(bitstring)):
        blocks[i//8]+=(bitstring[i])
    v1, v2, v3, v4 = 0, 0, 0, 0
    for block in blocks:
        lmax, lcurrent = 0, 0
        for i in range(len(block)-1):
            if(block[i]!=block[i+1]):
                lmax = max(lmax, lcurrent)
                lcurrent = 0
            else:
                lcurrent += 1
        if (lmax <= 1):
            v1 += 1
        elif (lmax == 2):
            v2 += 1
        elif (lmax == 3):
            v3 += 1
        else:
            v4 += 1
    chi = ((v1-16*0.2148)**2)/(16*0.2148) + ((v2-16*0.3672)**2)/(16*0.3672) + ((v3-16*0.2305)**2)/(16*0.2305) + ((v4-16*0.1875)**2)/(16*0.1875)
    pvalue = gammaincc(1.5,(chi/2))
    return pvalue
```

Результаты тестов:

| Язык ГПСЧ | Част. побит. тест | Тест на одинак. подряд идущие биты | Тест на самую длинную послед. ед. в блоке |
|---|---|---|---|
| Python | 0.5959 | 0.7045 | 0.1068 |
| C++ | 0.1116 | 0.3932 | 0.9737 |
| Java | 0.1572 | 0.8575 | 0.00103 |

## 3. Выводы по результатам тестирования

По результатам тестирования ГПСЧ на Java не прошёл тест на самую длинную последовательность единиц в блоке. Самым надёжным является ГПСЧ на Python.